LAPORAN PRAKTIKUM

PEMROGRAMAN MOBILE

Pertemuan 1: Pengenalan Flutter, Instalasi, dan Aplikasi Pertama

Nama	:	Muhamad Dika Ramadhan
NIM	:	20240801045
Program Studi	:	Teknik Informatika
Mata Kuliah	:	Pemrograman Mobile
Pertemuan	:	1

1. Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Menjelaskan apa itu Flutter dan Dart serta perbedaannya dengan pengembangan native.
2. Memasang Flutter SDK, editor, dan emulator/perangkat fisik.
3. Membuat dan menjalankan proyek Flutter pertama.
4. Memahami struktur folder proyek dan konsep dasar widget.
5. Memodifikasi UI sederhana dan menggunakan hot reload.

2. Landasan Teori

Flutter adalah UI toolkit buatan Google yang dipakai untuk membangun aplikasi mobile, web, dan desktop dari satu basis kode. Bahasa pemrograman yang digunakan adalah Dart. Berbeda dengan pengembangan native (Kotlin/Java untuk Android dan Swift untuk iOS) yang membutuhkan kode terpisah untuk setiap platform, Flutter cukup ditulis sekali lalu dijalankan di banyak platform.

Konsep utama yang dipelajari pada pertemuan ini:

- **Widget** : semua elemen di Flutter (teks, tombol, layout, bahkan aplikasi itu sendiri) adalah widget. Widget disusun bertingkat membentuk widget tree.
- **StatelessWidget** : widget yang tampilannya tidak berubah setelah dibuat.
- **StatefulWidget** : widget yang tampilannya bisa berubah saat aplikasi berjalan, melalui pemanggilan setState().

- **Hot Reload** : perubahan kode langsung tampil di aplikasi tanpa harus me-restart aplikasi dari awal.

3. Alat dan Bahan

- Laptop (RAM minimal 8 GB, ruang disk minimal 10 GB)
- Flutter SDK versi stabil terbaru
- Android Studio / VS Code dengan ekstensi Flutter & Dart
- Emulator Android (Pixel) atau HP Android dengan USB debugging aktif
- Koneksi internet

4. Langkah dan Hasil Praktikum

Bagian A. Instalasi dan Verifikasi

Langkah yang dilakukan: mengunduh Flutter SDK dari docs.flutter.dev lalu mengekstraknya dan menambahkan folder flutter/bin ke PATH; memasang Android Studio beserta Android SDK dan membuat emulator lewat Device Manager; memasang ekstensi Flutter dan Dart di editor; kemudian menjalankan dua perintah berikut di terminal.

```
flutter doctor
flutter doctor --android-licenses
```

Perintah	Fungsi
flutter doctor	Memeriksa kelengkapan lingkungan pengembangan (Flutter SDK, Android toolchain, editor, perangkat). Tanda centang hijau berarti siap dipakai, tanda silang harus diperbaiki.
flutter doctor --android-licenses	Menampilkan dan menerima lisensi Android SDK. Tanpa menerima lisensi, aplikasi tidak bisa dibuild untuk Android.

Bagian B. Membuat Proyek Pertama

```
flutter create praktikum_1
cd praktikum_1
flutter run
```

Perintah	Fungsi
flutter create praktikum_1	Membuat proyek Flutter baru bernama praktikum_1 lengkap dengan struktur folder dan contoh aplikasi counter bawaan.
cd praktikum_1	Berpindah ke folder proyek yang baru dibuat.

Perintah	Fungsi
flutter run	Membangun dan menjalankan aplikasi di emulator atau perangkat yang dipilih.

Hasilnya, aplikasi counter bawaan tampil di emulator dan tombol + berhasil menambah angka.

Bagian C. Struktur Proyek

Folder / File	Fungsi
lib/main.dart	Titik masuk aplikasi; seluruh kode utama ditulis di sini.
pubspec.yaml	Konfigurasi proyek: nama, dependensi (package), dan aset seperti gambar atau font.
android/, ios/	Kode platform native untuk Android dan iOS.
test/	Berkas pengujian otomatis.

Bagian D. Aplikasi "Hello Flutter"

Seluruh isi lib/main.dart diganti dengan kode berikut:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 1',
      home: Scaffold(
        appBar: AppBar(title: const Text('Hello Flutter')),
        body: const Center(
          child: Text(
            'Halo, nama saya Muhamad Dika Ramadhan!',
            style: TextStyle(fontSize: 24),
          ),
        ),
      ),
    );
  }
}
```

Penjelasan setiap fungsi dan widget:

Bagian Kode	Penjelasan
import 'package:flutter/material.dart'	Mengimpor pustaka Material Design Flutter yang berisi widget siap pakai seperti MaterialApp, Scaffold, dan Text.
void main()	Fungsi pertama yang dijalankan Dart ketika aplikasi dimulai.
runApp()	Menjalankan aplikasi dengan menjadikan widget yang diberikan (MyApp) sebagai akar widget tree.
class MyApp extends StatelessWidget	Widget utama aplikasi. Bersifat stateless karena tampilannya tetap.
const MyApp({super.key})	Constructor konstan; super.key meneruskan key ke kelas induk agar Flutter dapat mengidentifikasi widget.
@override	Penanda bahwa method di bawahnya menimpa method milik kelas induk.
Widget build(BuildContext context)	Method wajib yang mengembalikan susunan widget (UI). BuildContext menyimpan posisi widget di dalam tree.
MaterialApp	Widget akar yang menyediakan tema, navigasi, dan pengaturan Material Design. Properti title memberi nama aplikasi dan home menentukan halaman awal.
Scaffold	Kerangka halaman Material: menyediakan slot untuk appBar, body, floatingActionButton, dan lainnya.
AppBar(title: ...)	Bilah judul di bagian atas layar.
Center	Menempatkan child tepat di tengah area yang tersedia.
Text	Menampilkan teks. Properti style menerima TextStyle.
TextStyle(fontSize: 24)	Mengatur gaya teks, di sini ukuran huruf 24.
const	Menandai widget yang tidak berubah sehingga dibuat sekali saja. Ini menghemat memori dan mempercepat rebuild.

Susunan widget tree: MaterialApp → Scaffold → AppBar + Center → Text. Setelah teks [NAMA] diganti dengan nama sendiri dan file disimpan, hot reload langsung memperbarui tampilan tanpa restart aplikasi.

Bagian E. Widget Layout Dasar (Column)

Bagian body diganti dengan Column agar beberapa widget tersusun vertikal:

```
body: Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: const [
      Icon(Icons.flutter_dash, size: 80, color: Colors.blue),
```

```
      SizedBox(height: 16),
      Text('Halo, nama saya Muhamad Dika Ramadhan!',
          style: TextStyle(fontSize: 24)),
      Text('NIM: 20240801045'),
    ],
  ),
),
```

Bagian Kode	Penjelasan
Column	Widget layout yang menyusun children dari atas ke bawah (sumbu vertikal).
mainAxisAlignment: MainAxisAlignment.center	Mengatur perataan sepanjang sumbu utama. Pada Column sumbu utamanya vertikal, sehingga isi berada di tengah layar.
children	Daftar widget yang ditampilkan di dalam Column.
Icon(Icons.flutter_dash, size: 80, color: Colors.blue)	Menampilkan ikon bawaan Material (logo Dash) berukuran 80 dengan warna biru.
SizedBox(height: 16)	Kotak kosong setinggi 16 piksel yang berfungsi sebagai jarak antar widget.
Text(...)	Menampilkan nama dan NIM.

Bagian F. Widget Interaktif (StatefulWidget)

Class CounterPage ditambahkan di bawah MyApp, kemudian properti home diubah menjadi const CounterPage().

```
class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  State<CounterPage> createState() => _CounterPageState();
}

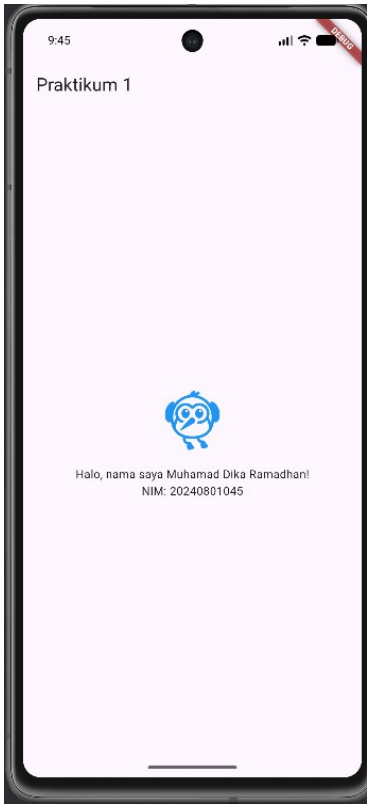
class _CounterPageState extends State<CounterPage> {
  int _count = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Counter Saya')),
      body: Center(
        child: Text('$_count',
            style: const TextStyle(fontSize: 48)),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => setState(() => _count++),
        child: const Icon(Icons.add),
```

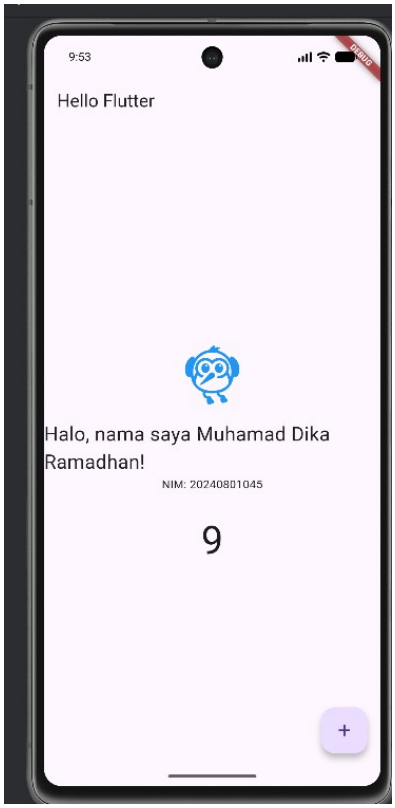
```
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
class CounterPage extends StatefulWidget	Widget yang tampilannya dapat berubah. Class ini hanya menyimpan konfigurasi, sedangkan data yang berubah disimpan di class State.
createState()	Method yang membuat objek State untuk widget ini, yaitu _CounterPageState.
class _CounterPageState extends State<CounterPage>	Tempat menyimpan state dan logika. Tanda _ di depan nama berarti private (hanya dapat diakses dalam file ini).
int _count = 0	Variabel state penampung nilai counter, awalnya 0.
build()	Dipanggil ulang setiap kali state berubah untuk menggambar ulang UI.
Text('$_count')	String interpolation: nilai _count disisipkan ke dalam teks.
FloatingActionButton	Tombol bulat melayang di pojok kanan bawah untuk aksi utama.
onPressed: () => setState(() => _count++)	Callback saat tombol ditekan. setState() memberi tahu Flutter bahwa state berubah sehingga build() dijalankan ulang dan angka baru tampil.
Icon(Icons.add)	Ikon tambah (+) di dalam tombol.

Hasil Praktikum Bagian D–F



Gambar 1. Tampilan nama, NIM, dan ikon (Bagian D–E)



Gambar 2. Tampilan dengan counter dan tombol + (Bagian F)

Pada Gambar 1 aplikasi menampilkan ikon, nama, dan NIM yang tersusun vertikal di tengah layar. Pada Gambar 2 ditambahkan angka counter dan tombol + di pojok kanan bawah; angka bertambah setiap tombol ditekan (pada tangkapan layar nilainya 9).

5. Latihan Mandiri

Latihan mandiri meliputi: (1) mengubah warna AppBar dan teks, (2) menambah tombol kurang (Icons.remove), (3) menambah tombol reset, dan (4) mencegah angka menjadi negatif. Berikut kode yang digunakan:

```
class _CounterPageState extends State<CounterPage> {
  int _count = 0;

  void _increment() => setState(() => _count++);

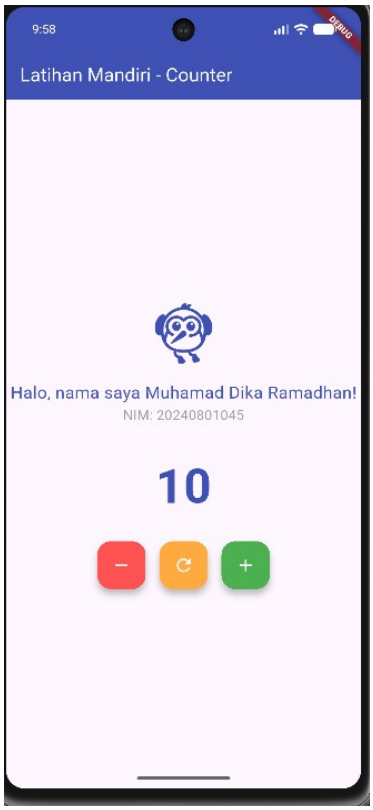
  void _decrement() {
    if (_count > 0) {          // cegah angka negatif
      setState(() => _count--);
    }
  }

  void _reset() => setState(() => _count = 0);
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Latihan Mandiri - Counter'),
        backgroundColor: Colors.indigo,
        foregroundColor: Colors.white,
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(Icons.flutter_dash,
                size: 80, color: Colors.indigo),
            const SizedBox(height: 16),
            const Text('Halo, nama saya Muhamad Dika Ramadhan!',
                style: TextStyle(fontSize: 20,
                    color: Colors.indigo)),
            const Text('NIM: 20240801045',
                style: TextStyle(color: Colors.grey)),
            const SizedBox(height: 32),
            Text('$_count',
                style: const TextStyle(fontSize: 56,
                    fontWeight: FontWeight.bold,
                    color: Colors.indigo)),
            const SizedBox(height: 32),
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                ElevatedButton(
                  onPressed: _decrement,
                  style: ElevatedButton.styleFrom(
                      backgroundColor: Colors.red),
                  child: const Icon(Icons.remove),
                ),
                const SizedBox(width: 16),
                ElevatedButton(
                  onPressed: _reset,
                  style: ElevatedButton.styleFrom(
                      backgroundColor: Colors.orange),
                  child: const Icon(Icons.refresh),
                ),
                const SizedBox(width: 16),
                ElevatedButton(
                  onPressed: _increment,
                  style: ElevatedButton.styleFrom(
                      backgroundColor: Colors.green),
                  child: const Icon(Icons.add),
                ),
              ],
            ),
          ],
        ),
      ),
    ),
```

```
    );
  }
}
```

Bagian Kode	Penjelasan
_increment()	Menambah _count sebanyak 1 di dalam setState().
_decrement()	Mengurangi _count sebanyak 1. Pengecekan if (_count > 0) mencegah angka menjadi negatif.
_reset()	Mengembalikan _count ke 0.
backgroundColor / foregroundColor	Mengatur warna latar AppBar (indigo) dan warna teks/ikon di atasnya (putih).
TextStyle(color:, fontWeight:)	Mengubah warna teks dan menebalkan angka counter.
Row	Layout yang menyusun widget secara horizontal; dipakai untuk menjajarkan ketiga tombol.
ElevatedButton	Tombol bertimbul. onPressed menentukan fungsi yang dijalankan dan ElevatedButton.styleFrom mengatur warna tombol.
SizedBox(width: 16)	Jarak horizontal antar tombol.



Gambar 3. Hasil Latihan Mandiri: counter dengan tombol kurang, reset, dan tambah

Tombol merah mengurangi angka (tidak akan turun di bawah 0), tombol oranye mereset angka ke 0, dan tombol hijau menambah angka. Warna AppBar dan teks sudah diubah menjadi indigo.

6. Tugas: Kartu Perkenalan

Tugas berupa aplikasi satu halaman yang menampilkan ikon/foto, nama, NIM, jurusan, dan hobi menggunakan Column, Text, Icon, dan SizedBox.

```
class KartuPerkenalan extends StatelessWidget {
  const KartuPerkenalan({super.key});

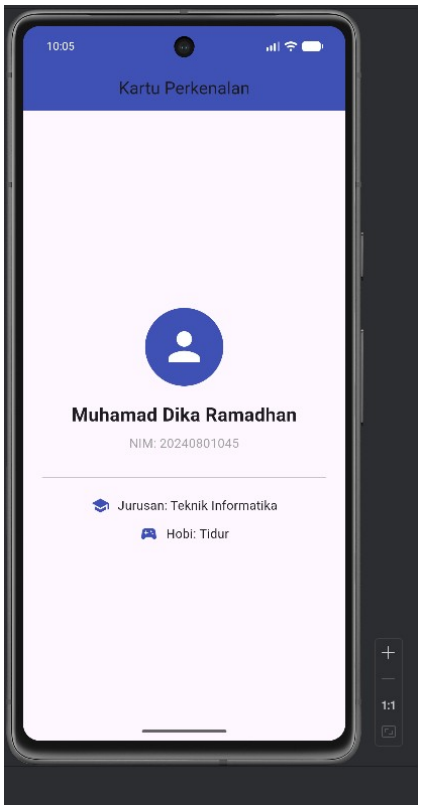
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Kartu Perkenalan'),
        backgroundColor: Colors.indigo,
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const CircleAvatar(
              radius: 40,
              backgroundColor: Colors.indigo,
              child: Icon(Icons.person,
                  size: 50, color: Colors.white),
            ),
            const SizedBox(height: 16),
            const Text('Muhamad Dika Ramadhan',
                style: TextStyle(fontSize: 20,
                    fontWeight: FontWeight.bold)),
            const SizedBox(height: 4),
            const Text('NIM: 20240801045',
                style: TextStyle(color: Colors.grey)),
            const SizedBox(height: 24),
            const Divider(indent: 40, endIndent: 40),
            const SizedBox(height: 8),
            const _InfoRow(icon: Icons.school,
                text: 'Jurusan: Teknik Informatika'),
            const SizedBox(height: 8),
            const _InfoRow(icon: Icons.sports_esports,
                text: 'Hobi: Tidur'),
          ],
        ),
      ),
    );
  }
}

class _InfoRow extends StatelessWidget {
  final IconData icon;
  final String text;
```

```
  const _InfoRow({required this.icon, required this.text});

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Icon(icon, color: Colors.indigo),
        const SizedBox(width: 8),
        Text(text),
      ],
    );
  }
}
```

Bagian Kode	Penjelasan
KartuPerkenalan (StatelessWidget)	Halaman kartu perkenalan. Stateless karena datanya tetap.
CircleAvatar	Widget berbentuk lingkaran untuk foto profil; di sini berisi ikon orang (Icons.person).
Column + mainAxisAlignment.center	Menyusun semua elemen secara vertikal di tengah layar.
Text dengan FontWeight.bold	Menampilkan nama dengan huruf tebal.
Divider	Garis pemisah tipis antara identitas dan informasi tambahan. indent dan endIndent memberi jarak kiri dan kanan.
_InfoRow	Widget buatan sendiri yang dapat dipakai ulang. Menerima parameter icon dan text, lalu menampilkannya dalam satu baris.
required this.icon	Parameter wajib diisi saat _InfoRow dipanggil.
IconData	Tipe data untuk ikon Flutter, misalnya Icons.school dan Icons.sports_esports.



Gambar 4. Hasil aplikasi Kartu Perkenalan

Aplikasi menampilkan avatar, nama, NIM, jurusan, dan hobi tersusun rapi di tengah layar sesuai ketentuan tugas.

7. Jawaban Pertanyaan Refleksi

1. Apa perbedaan StatelessWidget dan StatefulWidget?

StatelessWidget adalah widget yang tampilannya tidak berubah setelah dibuat, karena tidak memiliki data yang berubah-ubah (contoh: MyApp dan KartuPerkenalan). StatefulWidget adalah widget yang memiliki state sehingga tampilannya dapat berubah saat aplikasi berjalan, misalnya ketika pengguna menekan tombol (contoh: CounterPage). StatefulWidget terdiri dari dua class, yaitu widget dan State-nya.

2. Mengapa perubahan variabel _count perlu dibungkus setState()?

Karena setState() memberi tahu Flutter bahwa state telah berubah dan UI perlu digambar ulang dengan memanggil build() lagi. Jika _count diubah tanpa setState(), nilai variabelnya memang berubah tetapi layar tidak diperbarui sehingga angka yang tampil tetap sama.

3. Apa keuntungan hot reload dibanding rebuild penuh?

Hot reload menyuntikkan perubahan kode ke aplikasi yang sedang berjalan dalam hitungan detik tanpa restart, sehingga hasil perubahan UI langsung terlihat dan state aplikasi (misalnya nilai counter) tetap dipertahankan. Rebuild penuh membutuhkan waktu jauh lebih lama karena aplikasi

harus dikompilasi dan dijalankan ulang dari awal. Hot reload membuat proses eksperimen dan perbaikan tampilan jauh lebih cepat.

8. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa Flutter memungkinkan pembuatan aplikasi dengan satu basis kode menggunakan bahasa Dart, di mana seluruh tampilan dibangun dari widget yang disusun dalam widget tree. StatelessWidget digunakan untuk tampilan tetap, sedangkan StatefulWidget bersama setState() digunakan untuk tampilan yang interaktif. Widget layout seperti Column dan Row, serta widget dasar seperti Text, Icon, dan SizedBox, sudah cukup untuk membuat aplikasi sederhana seperti counter dan kartu perkenalan. Fitur hot reload sangat membantu mempercepat proses pengembangan.